HTTP API Reference

Routes, parameters, and response shapes

Hamza Abdul Karim · F23607046 — May 2026

ABSTRACT

The Flask application exposes both HTML pages and a small JSON API. This document enumerates every route, lists their parameters with default values, and gives the schema of the JSON responses.

1. Routes

Method	Path	Description
GET	/	Landing page with search form, examples, feature grid.
GET	/about	Project overview, API summary, links.
GET	/search	HTML results page.
GET	/api/search	Ranked results as JSON.
GET	/api/stats	Corpus statistics as JSON.
GET	/healthz	Liveness probe.

2. GET /api/search

Run a query and return ranked results as JSON. The route parses bare terms, required terms (`+term`), excluded terms (`-term`), quoted phrases, and the boolean operators `AND` / `OR` / `NOT`.

2.1 Query parameters

Name	Type	Default	Description
q	string	required	Raw query text.
model	string	bm25	One of bm25 or tfidf.
k	int	10	Number of results, clamped to 1..50.

2.2 Example

```
curl "http://127.0.0.1:5000/api/search?q=machine+learning&model=bm25&k=3"
```

2.3 Response shape

```
{
"query": "machine learning",
"model": "bm25",
"top_k": 3,
"results": [
{
"doc_id": 1,
"title": "machine learning",
"snippet": "<mark>Machine</mark> <mark>learning</mark> ...",
"score": 3.59904,
"source": "machine_learning.txt"
}
],
"suggestion": null
}
```

If the query returns no results, the `suggestion` field contains a spell-corrected variant produced by the Damerau–Levenshtein checker; otherwise it is null.

3. GET /api/stats

Returns counters describing the loaded index.

```
{
"documents": 7,
"vocabulary_size": 305,
"total_tokens": 482,
"avg_doc_length": 68.85,
"model": "bm25"
}
```

4. GET /healthz

Lightweight liveness check, suitable for Docker or Kubernetes probes.

```
{
"status": "ok",
"documents": 7
}
```

5. Error Handling

Condition	Response
Unknown model	400 with {"error": "unknown model 'X'"}
Missing q	200 with empty results
Out-of-range k	Silently clamped to [1, 50]

6. Calling the API

6.1 Python

```
import requests

resp = requests.get(
"http://127.0.0.1:5000/api/search",
params={"q": "neural networks", "model": "tfidf", "k": 5},
timeout=5,
)
for hit in resp.json()["results"]:
print(f"{hit[\"score\"]:6.3f} {hit[\"title\"]}")
```

6.2 JavaScript

```
const res = await fetch(
'/api/search?' + new URLSearchParams({q: 'neural', model: 'bm25'})
);
const data = await res.json();
console.log(data.results);
```